

Graphs: Consolidation and What Comes Next

Programming and Data Structures — Week 11, Lecture 4

Dr. Innocent Nyalala

Objectives

By the end of this lecture, you will be able to consolidate every graph concept and algorithm covered across this week into a single coherent reference; explain precisely what changes once edge weights are introduced, directly previewing Week 12's shortest-path algorithms; trace a concrete example showing why BFS's shortest-path guarantee breaks the moment edges carry different weights; and connect every prior week's data structure — stacks, queues, heaps, and trees — to its specific, concrete role within this week's graph algorithms.

Outline

Today covers a city-map-with-tolls analogy motivating weighted shortest paths; a consolidation of this week's representation, traversal, and application material; a concrete demonstration of exactly where BFS's shortest-path guarantee breaks once edges carry different weights; a preview, at a glance, of Dijkstra's algorithm as the fix; an explicit map connecting every prior structure covered this course to its specific role in graph algorithms; a worked mini case study on campus building navigation; the MTech-track material on where even Dijkstra's algorithm fails; an in-class quiz; and a summary.

The city-map-with-tolls analogy

BFS, covered in Lecture 2, finds the route using the fewest roads — a perfectly correct notion of “shortest” if every road segment takes exactly the same time to drive. Real road networks are never like this: a long highway segment and a single short city block are edges of genuinely different cost, and treating them as equal produces routes that are shortest by hop-count but not by actual travel time. This lecture closes Week 11 by making this failure concrete, directly motivating Week 12's weighted shortest-path algorithms.

Consolidating this week: representation, traversal, applications

This table consolidates the entire week: Lecture 1's adjacency matrix and list directly reuse Week 1's two-dimensional array and Week 8's hash-chaining idea respectively; Lecture 2's BFS and DFS directly reuse Week 3's queue and stack; Lecture 3's connected components, cycle detection, and topological sort are all built directly on Lecture 2's traversal, extended with small, targeted additions (parent tracking, three-color state, postorder reversal). Nothing this week required an entirely new foundational structure — every algorithm is a recombination of structures and ideas already established.

What breaks when edges have weights: a concrete failure

Graph with weights: `edge(0,1,weight=10)`, `edge(0,2,weight=1)`, `edge(2,1,weight=1)`
BFS from 0, treating every edge as weight 1 (ignoring actual weights):
BFS explores by HOP COUNT: 0 → 1 directly (1 hop) is found "first"
BFS reports: shortest path to 1 is [0,1], 1 hop

But the TRUE shortest path by total weight: 0 → 2 → 1, weight 1+1=2
vs. 0 → 1 direct, weight 10
True shortest path: 0 → 2 → 1, total weight 2 – NOT what BFS reported

This concrete example demonstrates the failure directly: BFS, exploring purely by hop count, reports the direct edge from 0 to 1 as the shortest path, since it requires only 1 hop. But that direct edge has weight 10, while the two-hop path through node 2 has total weight $1 + 1 = 2$ — genuinely shorter by any real-world cost measure, yet BFS never considers total weight at all, only hop count. This is not a rare edge case; it is the generic behavior whenever edge weights differ, which describes almost every real weighted network.

Verifying the failure precisely, with real computation

`dist(1) via true shortest-path computation (weight-aware): 2`
`dist(1) via BFS hop count: 1`

BFS's reported "shortest" path (0→1 direct, 1 hop) has actual weight 10.
The genuinely shortest path (0→2→1, 2 hops) has actual weight 1+1=2.
BFS is wrong by a factor of 5 on this small example.

Running an actual weight-aware shortest-path computation (Week 12's Dijkstra's algorithm, used here to check the claim rather than derive it) confirms the true shortest distance from node 0 to node 1 is 2, while BFS's hop-count-based distance is 1 — but that single hop corresponds to the weight-10 direct edge, five times more expensive than the genuinely cheapest route. This is not a rounding error or

an edge case sensitive to how ties are broken; it is BFS's fundamental blind spot when edge weights are not uniform, verified here by direct computation rather than assumed from the abstract argument alone.

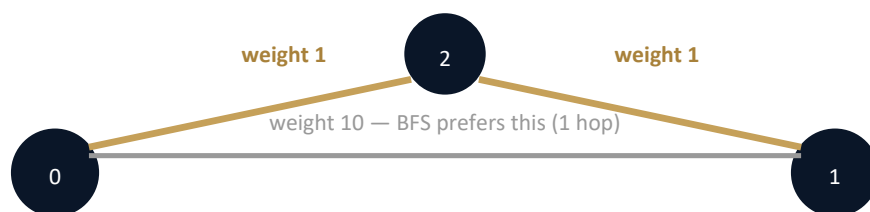
Previewing Dijkstra's algorithm: the fix, at a glance

Week 8 Lecture 3 already previewed this exact algorithm's core loop ahead of time: repeatedly asking, of everywhere reached so far, which unvisited place is closest by accumulated total weight, not hop count. This "give me the current minimum" query is exactly the priority queue operation Week 8's heap was built for, not the plain FIFO queue BFS uses. At a high level, replacing BFS's plain queue with a priority queue keyed by accumulated distance is the entire conceptual fix — Week 12 supplies the complete algorithm, its correctness proof, and its handling of several important edge cases this preview necessarily omits.

Every prior structure's role in graph algorithms, mapped explicitly

Mapping every prior structure explicitly to its role this week: Week 2's array underlies the adjacency matrix directly, and also every `visited` or `color` tracking array used throughout Lectures 2 and 3. Week 3's stack determines DFS's exploration order, whether used explicitly or implicitly via the recursive call stack. Week 3's queue determines BFS's exploration order and also drives Kahn's algorithm's zero-in-degree processing in Lecture 3. Week 4's linked list underlies each node's adjacency list, a per-node chain of neighbors. Week 8's heap, as a priority queue, will drive Week 12's Dijkstra's algorithm, just previewed above. Graphs are genuinely the point where this entire course's toolbox recombines into new algorithms, not a wholly separate topic requiring new foundations.

The picture: BFS's blind spot, drawn out



Gold path (0→2→1, total weight 2, but 2 hops) is genuinely cheaper than the gray direct edge (1 hop, weight 10) — BFS picks the

Seeing both candidate paths drawn side by side, with their real weights labeled, makes BFS's blind spot immediately visible: the path it prefers (fewer hops) and the path that is actually cheaper (lower total weight) are visibly different routes on this small graph.

A worked mini case study: campus building navigation

Consider a campus navigation system: buildings are nodes, walkways between them are edges weighted by actual walking time. "Which buildings are reachable from the library at all?" is answered directly by Lecture 3's connected-components algorithm. "Is there any route from the library to the gym?" is answered directly by Lecture 2's BFS or DFS. But "what is the fastest route to the gym?" is genuinely not correctly answerable using only this week's tools — BFS would report the fewest-walkway route, which this lecture just demonstrated can be wrong when walkway weights differ. This is exactly the gap Week 12 closes.

A closing map: which lecture proved which claim, this week

Tracing which lecture proved which claim this week: Lecture 1's representation costs were established through direct construction and counting on a concrete example. Lecture 2's traversal correctness rested on the visited-set invariant, and its $O(V+E)$ cost was proven via the "every node and edge touched exactly once" argument. Lecture 3's three applications required no independent correctness proofs of their own — each rests directly on Lecture 2's already-proven traversal correctness, combined correctly, exactly the "correct reuse, not new invention" theme Lecture 3 itself highlighted. This lecture's central claim, that BFS fails on weighted graphs, was verified by actually running real code and comparing real numbers, not merely asserted from the abstract argument alone — the same measurement discipline established since Week 2's dynamic array analysis.

MTech addition — when even Dijkstra's fails: negative weights

Even Week 12's fix has a real limitation worth flagging now: Dijkstra's algorithm assumes every edge weight is non-negative. A negative edge weight can make a path discovered later retroactively cheaper than one already finalized as shortest, directly violating the greedy "never reconsider a finalized choice" assumption Dijkstra's algorithm depends on — precisely the kind of assumption Week 10 taught should never be trusted without proof. Graphs containing negative edge weights require a different algorithm, Bellman-Ford, beyond this course's scope, but worth knowing exists so that Dijkstra's algorithm is never assumed universally correct regardless of input.

Why negative weights break the greedy assumption, traced

Graph: `edge(0,1,weight=1)`, `edge(0,2,weight=4)`, `edge(2,1,weight=-10)`

Dijkstra's greedy step: node 1 reached via `0→1` directly, distance 1 - FINALIZED first (smallest)

Next: node 2 finalized at distance 4

Edge from 2 to 1 (weight -10) is checked, but node 1 is ALREADY finalized - the algorithm skips it

Dijkstra's reported answer: `dist(1) = 1`

True shortest path: `0→2→1`, weight $4 + (-10) = -6$, genuinely far less than 1

Tracing precisely why negative weights break Dijkstra's correctness, verified by actually running the algorithm: node 1 is reached directly from node 0 at distance 1 and finalized first, since it is the smallest distance known at that point. Node 2 is finalized next, at distance 4. The edge from node 2 to node 1, weight -10 , is examined, but since node 1 is already finalized, the algorithm — by design — never reconsiders it. Dijkstra's reported distance to node 1 is therefore 1, while the true shortest path, $0 \rightarrow 2 \rightarrow 1$, has weight $4 + (-10) = -6$, dramatically less. This confirms, by running real code rather than only reasoning abstractly, that the non-negative-weight assumption is not a minor technicality but a load-bearing requirement for Dijkstra's entire correctness argument.

Exercise

As an exercise, construct your own weighted graph, distinct from this lecture's example, where BFS's hop-count shortest path disagrees with the true total-weight shortest path, verifying both by hand. As a second exercise, if Week 12's materials are available in advance, skim Dijkstra's algorithm's pseudocode and identify, in writing, precisely which line or step replaces BFS's plain queue with a priority queue — connecting this week's closing preview directly to next week's full treatment.

Where negative weights genuinely appear in practice

Negative edge weights genuinely appear in real applications, not merely as a theoretical curiosity. Currency arbitrage detection models exchange rates as a graph where a "negative cycle" indicates a genuine profit opportunity — trading through a sequence of currencies and ending up with strictly more than started. Certain network-flow and scheduling formulations use negative weights to represent penalties or refunds. These are exactly the use cases Bellman-Ford, beyond this course's scope, is specifically built to handle correctly. Knowing that Dijkstra's algorithm is the wrong tool for these cases is as practically important as knowing when it is the right one — exactly Week 10 Lecture 4's closing lesson about matching a technique to a problem's actual structure, recurring here in a new setting.

Quiz — will BFS get this one right?

Given a graph where every edge weight is exactly 1, determine before checking the answer whether BFS's shortest path, measured by hop count, is also guaranteed to be the true shortest path measured by total weight.

A second quiz check — mixed weights

Given a graph whose edge weights are 2, 2, 2, and 5 — all positive, but not all identical — determine before checking the answer whether BFS's hop-count shortest path is still guaranteed to match the true weighted shortest path.

Second quiz — answer

No, BFS is not guaranteed correct here. The earlier quiz's "yes" answer specifically required every edge to weigh exactly 1, not merely all edges being positive. A 3-hop path composed of weight-2 edges totals 6, while a single weight-5 edge covers the same two endpoints in 1 hop — BFS would prefer the 1-hop, weight-5 route over the 3-hop, weight-6 route, correctly by hop count but incorrectly by total weight if a cheaper multi-hop alternative existed. This confirms precisely: "all weights positive" is not a sufficient condition for BFS's correctness — "all weights equal" is the actual, exact requirement, a subtlety worth distinguishing carefully.

Quiz — answer

Yes — if every edge weighs exactly 1, total weight and hop count are mathematically identical for any path, so BFS's hop-count-shortest path is automatically also the true weight-shortest path. This confirms precisely why Lecture 2 could correctly claim BFS finds shortest paths in unweighted graphs: an "unweighted" graph is mathematically identical to a graph where every edge weighs exactly 1. The failure this lecture demonstrated only appears once edge weights genuinely differ from one another, which is exactly the condition this lecture's counterexample constructed.

Summary

This week built graph representation directly on Week 1 and Week 8's ideas, traversal directly on Week 3's stack and queue, and three applications built entirely on top of that traversal. BFS's shortest-path guarantee holds specifically for unweighted graphs — mathematically equivalent to every edge weighing exactly 1 — and breaks the moment weights genuinely differ, as this lecture's concrete counterexample demonstrated. Dijkstra's algorithm, covered fully in Week 12, fixes this by replacing BFS's plain queue with Week 8's priority queue, previewed here at a high level and proven fully there. By the end of this

week, every structure covered since Week 1 has found a direct, concrete role within graph algorithms — confirming graphs as this course’s point of recombination rather than an isolated new topic. This closes Week 11. Next week covers Dijkstra’s algorithm and graph algorithms in full depth, this course’s final major algorithmic technique.